



Universidad Nacional Autónoma de México

FACULTAD DE CIENCIAS

Heurísticas de Optimización Combinatoria

Grupo: 7167 - Semestre: 2027-1

Lectura 01

Problema del Agente Viajero mediante

Aceptación por Umbrales

FECHA DE ENTREGA: 18/09/2026

Profesor:

Canek Peláez Valdés

Alumna:

Carpinteiro Tapia Mariam Ximena

1. Abstract

Presentamos la implementación de la heurística de Aceptación por Umbrales (*Threshold Accepting*) para resolver instancias del Problema del Agente Viajero (TSP). Dada una gráfica no completa de ciudades con distancias geográficas, se busca la trayectoria de menor costo que visite un subconjunto de ellas, penalizando las aristas inexistentes para favorecer soluciones factibles. El sistema se implementó en C++ con una función de costo evaluable en tiempo constante por vecino mediante evaluación incremental, y se paralelizó la experimentación sobre miles de semillas. Para las instancias de prueba de 40 y 150 ciudades se obtuvieron soluciones factibles con costos normalizados de 0.2659 y 0.1400 respectivamente.

2. Introducción

El Problema del Agente Viajero (TSP, por sus siglas en inglés) es uno de los problemas clásicos de optimización combinatoria y pertenece a la clase de problemas NP-duros: no se conoce un algoritmo que encuentre la solución óptima en tiempo polinomial. Por ello, para instancias de tamaño no trivial se recurre a heurísticas, las cuales dejan un poco de lado la optimalidad a cambio de encontrar soluciones suficientemente buenas en un tiempo razonable.

En este trabajo se aborda una variante del TSP: dado un subconjunto de ciudades sobre un mapa, se busca la trayectoria más corta que pase por todas ellas. La gráfica de ciudades es densa pero no completa, de modo que no toda trayectoria es válida; una parte central del problema es precisamente distinguir y favorecer las soluciones factibles.

Se implementa la heurística de Aceptación por Umbrales, una variante del Recocido Simulado propuesta por Dueck y Scheuer en 1990, y realizamos experimentación computacional para calibrar sus parámetros y evaluar la calidad de las soluciones obtenidas sobre dos instancias de prueba, de 40 y 150 ciudades.

3. El problema

Empecemos describiendo el problema, la instancia se modela mediante una gráfica ponderada $G(V, E)$, donde los vértices son ciudades y las aristas representan conexiones entre ellas, con un peso igual a la distancia geográfica. Esta gráfica es densa pero no completa, es decir que existen pares de ciudades sin conexión directa.

Sea $S \subseteq V$ el subconjunto de ciudades a visitar. Para simplificar la optimización se trabaja sobre la gráfica completa G_S inducida por S , con una función de peso aumentada w_S : si la arista existe en E se usa su distancia real; si no, se asigna un peso deliberadamente grande, igual a la distancia natural multiplicada por la distancia máxima de S . Este castigo hace que las soluciones con aristas inexistentes tengan un costo muy alto, de modo que la heurística tiende a descartarlas.

Cualquier permutación de S es una solución en G_S , pero se distinguen dos tipos: **factibles**, aquellas cuyas aristas consecutivas existen todas en E ; y **no factibles**, las que usan al menos una arista inexistente.

3.1. Distancia natural

Para las aristas que no aparecen en la base de datos, la distancia se calcula con la fórmula del gran círculo (*haversine*), suponiendo la Tierra una esfera de radio $R = 6,373,000$ metros. Dadas dos ciudades u y v con coordenadas en latitud y longitud:

$$d(u, v) = R \cdot 2 \arctan \left(\frac{\sqrt{A}}{\sqrt{1 - A}} \right),$$

donde A combina las diferencias de latitud y longitud convertidas a radianes.

3.2. Función de costo

El costo de una trayectoria $P = v_{\rho(1)}, \dots, v_{\rho(k)}$ es la suma de los pesos aumentados de sus aristas consecutivas, normalizada:

$$f(P) = \frac{\sum_{i=2}^k w_S(v_{\rho(i-1)}, v_{\rho(i)})}{N(S)}.$$

El normalizador $N(S)$ es la suma de las $k - 1$ aristas reales más pesadas entre ciudades de S . Con esta normalización, toda solución factible se evalúa en el intervalo $[0, 1]$, mientras que las no factibles superan 1.

4. La heurística: Aceptación por Umbrales

La Aceptación por Umbrales (*Threshold Accepting*), propuesta por Dueck y Scheuer en 1990, es una variante simplificada del Recocido Simulado. Parte de una solución inicial y una temperatura T , y de forma iterativa propone una solución vecina s' , aceptándola cuando

$$f(s') \leq f(s) + T.$$

Es decir, acepta toda mejora y también empeoramientos pequeños, acotados por el umbral T . Esta tolerancia controlada a empeorar es lo que nos permite escapar de mínimos locales.

La temperatura desciende de forma gradual, el descenso se organiza en **lotes**: un lote es un número fijo L de soluciones aceptadas. Se ejecutan lotes a una misma temperatura mientras el promedio de sus costos siga mejorando (equilibrio térmico); cuando deja de mejorar, la temperatura se enfría multiplicándola por un factor $\varphi \in (0, 1)$. El proceso termina cuando T cae por debajo de un cero virtual ε . A lo largo de toda la ejecución se conserva la mejor solución encontrada.

4.1. Vecindad

Dos soluciones son vecinas si una se obtiene de la otra intercambiando dos elementos de la permutación. Para una instancia de k ciudades existen $\binom{k}{2} = k(k - 1)/2$ vecinos posibles de cada

solución.

4.2. Temperatura inicial

La temperatura inicial se calcula mediante búsqueda binaria: se busca un valor de T para el cual se acepte un porcentaje P elevado de vecinos de un escenario inicial (se usó $P = 0.9$). Primero se acota un intervalo duplicando o reduciendo T a la mitad hasta rodear el objetivo, y luego se aplica búsqueda binaria sobre ese intervalo hasta acercarse a P .

5. Diseño de la solución

El sistema se estructuró en componentes con responsabilidades separadas:

- **Lectura y datos.** Un lector interpreta los archivos de instancia `.tsp`, y una capa de acceso a la base de datos SQLite recupera las ciudades y las conexiones. La gráfica real solo se consulta una vez, al inicio.
- **Instancia.** La clase `Instance` reindexa las ciudades (cuyos identificadores son dispersos) a índices contiguos $0, \dots, k - 1$, y construye una matriz de adyacencia $k \times k$ con los pesos aumentados ya calculados. La distancia máxima y el normalizador se calculan una única vez en el constructor, pues no dependen del orden de la trayectoria.
- **Solución.** La clase `Solution` representa una permutación con su costo, y sabe generar un vecino (intercambio de dos posiciones) y deshacerlo.
- **Heurística.** La clase `ThresholdAccepting` orquesta la búsqueda: lotes, enfriamiento y conservación de la mejor solución. No conoce los detalles de la gráfica ni de la base de datos.

Esta separación permite probar cada pieza de forma aislada mediante pruebas unitarias, y sustituir la heurística sin tocar el modelo del problema.

6. Detalles de implementación

El sistema se implementó en C++ (estándar C++17), usando SQLite para la base de datos de ciudades. Se destacan las siguientes decisiones:

- **Matriz de pesos con arreglos.** Los pesos aumentados w_S se precaculan en una matriz plana $k \times k$, de modo que evaluar una solución es solo acceso a arreglo, sin consultas a la base de datos ni cálculos trigonométricos en el ciclo caliente.
- **Evaluación incremental.** Un intercambio de dos posiciones altera a lo más cuatro aristas de la trayectoria. En lugar de recalcular la suma completa ($O(k)$), se restan las aristas viejas y se suman las nuevas ($O(1)$). Para acotar el error de punto flotante acumulado tras millones de intercambios, cada cierto número de operaciones se recalcula el costo completo.
- **Reproducibilidad.** Toda la aleatoriedad proviene de un generador Mersenne Twister con semilla explícita. Una corrida queda determinada por completo por su semilla y sus parámetros, lo que permite reproducir cualquier resultado y comparar configuraciones de forma justa.
- **Cláusula de escape.** Se implementó un mecanismo opcional de recalentamiento: si la mejor solución no mejora durante un número de lotes, la temperatura se reinicia para permitir escapar de un mínimo local. Este mecanismo aumentó notablemente la proporción de corridas que terminan en solución factible, pero no me proporcionaron el mejor costo.
- **Experimentación paralela.** Como cada semilla es independiente, los barridos de miles de semillas se ejecutan en paralelo, una por núcleo, reduciendo el tiempo total en proporción al número de núcleos disponibles. Además compilamos con optimización (-O2) lo cual redujo el tiempo por corrida.

7. Resultados

Se resolvieron las dos instancias de prueba, de 40 y 150 ciudades. Los mejores resultados se obtuvieron con los parámetros $L = 4000$, $\varphi = 0.995$, $\varepsilon = 10^{-5}$ y $P = 0.9$.

Instancia	Semilla	Costo	Factible
input-40	207	0.265907	Sí
input-150	337	0.140028	Sí

Tabla 1: Mejores soluciones encontradas para cada instancia.

Ambas soluciones son factibles, es decir, su costo normalizado es menor que 1 y todas sus aristas existen en la gráfica real. Como referencia, la trayectoria trivial (las ciudades en orden de identificador) no es factible en ninguna de las dos instancias, con evaluaciones del orden de millones.

7.1. Verificación de las soluciones

Las mejores soluciones se guardaron en archivos `.tsp` y se reevaluaron con el programa para confirmar su costo y factibilidad. Las figuras 1 y 2 muestran la salida del programa al reevaluar cada solución: el costo coincide con el reportado durante la búsqueda y ambas resultan factibles.

```
maia ~/Documents/bly/7th_sem/heurísticas/tsp/tsp-heurísticas → ./build/tsp-analyzer
mejor-40.tsp
  Filename: mejor-40.tsp
    Path: 657,168,1,7,816,982,332,820,654,490,653,2,656,815,496,172,163,329,493,97
          9,4,165,3,333,981,6,5,978,817,489,492,491,984,331,164,327,980,244,75,652
    Maximum: 4970123.962350250
  Normalizer: 181500915.901503146
  Evaluation: 0.265907346
  Feasible: YES
```

Figura 1: Reevaluación de la mejor solución de input-40 (costo 0.265907).

```
● maia ~/Documents/bly/7th_sem/heuristicas/tsp/tsp-heuristicas → ./build/tsp-analyzer
mejor-150.tsp
  Filename: mejor-150.tsp
    Path: 494,495,167,328,326,1037,169,1073,330,819,655,818,666,658,166,822,74,100
1,297,980,336,840,350,670,327,511,504,500,825,20,660,510,343,985,674,334,999,995,984
,17,444,826,244,339,1038,12,828,502,340,675,186,520,16,671,179,512,821,1075,483,652,
171,346,75,183,77,489,4,174,988,165,3,981,333,990,353,27,991,185,22,351,1004,6,5,978
,352,668,176,23,817,347,499,491,492,25,501,164,11,1003,349,331,662,8,837,979,493,509
,329,163,673,665,676,490,344,653,654,26,820,345,332,181,14,982,187,816,678,823,7,507
,656,2,173,667,184,815,172,182,496,839,19,505,168,508,986,9,1,829,661,832,663,657
  Maximum: 4978506.478459956
  Normalizer: 722598784.973403096
  Evaluation: 0.140027769
  Feasible: YES
```

Figura 2: Reevaluación de la mejor solución de input-150 (costo 0.140028).

7.2. Evolución del costo

Las figuras 3 y 4 muestran la evolución del costo de la solución aceptada a lo largo de las evaluaciones aceptadas. En ambas se aprecia el mismo comportamiento: un descenso pronunciado al inicio, cuando la temperatura es alta y la búsqueda explora aceptando incluso soluciones no factibles con penalizaciones grandes, seguido de un largo tramo en el que el costo se estabiliza cerca de su valor final conforme la temperatura disminuye. La caja en cada gráfica indica el costo de la última solución aceptada, que puede diferir ligeramente del mejor costo global, pues la heurística conserva por separado la mejor solución vista durante toda la corrida.

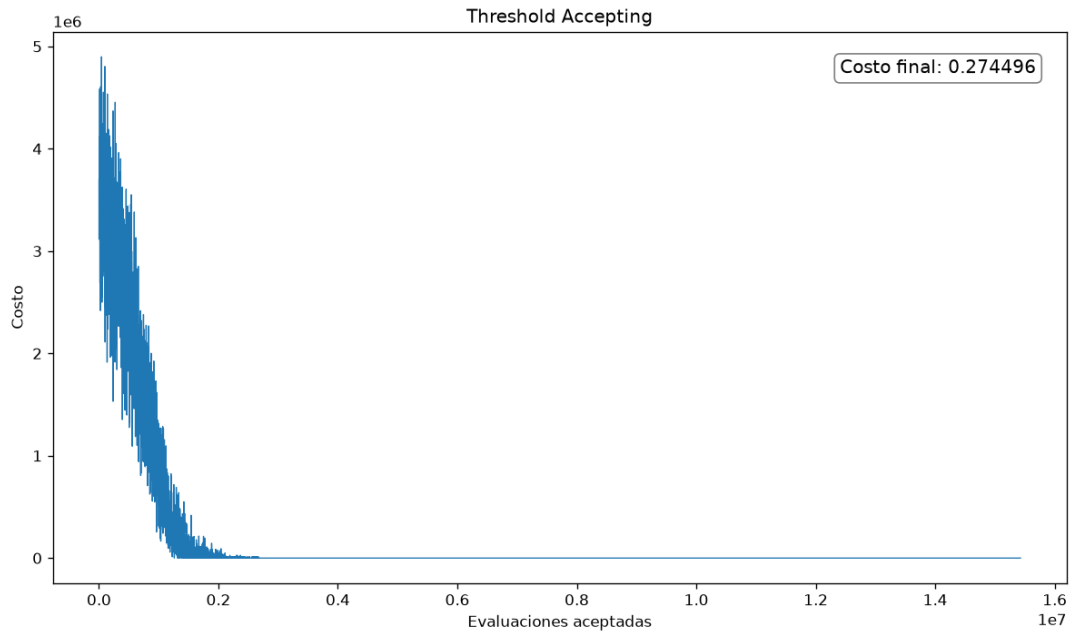


Figura 3: Evolución del costo para input-40 (semilla 207).

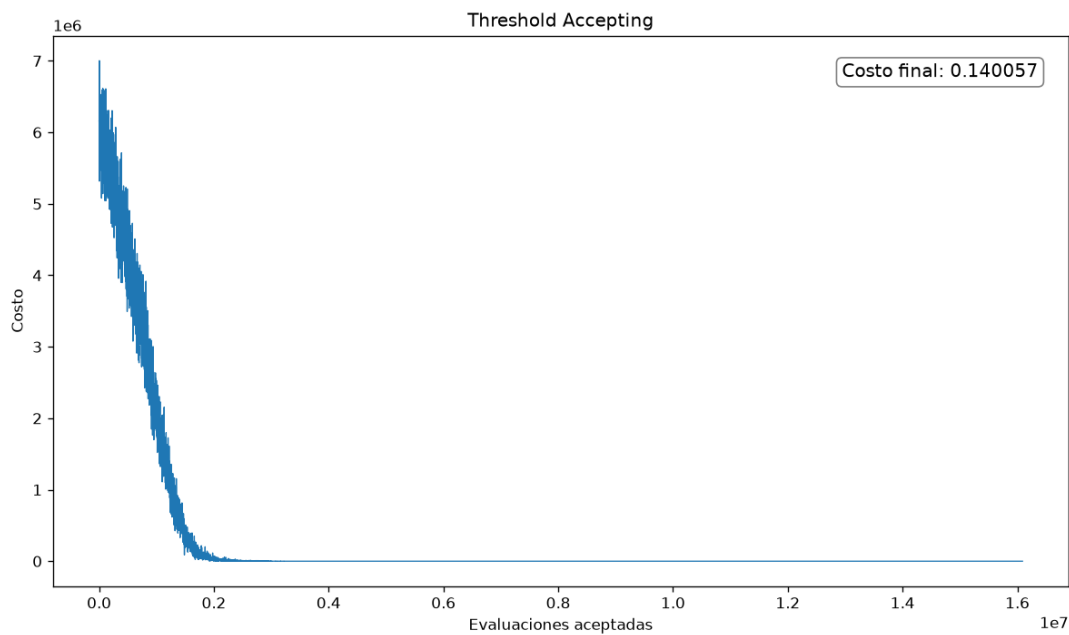


Figura 4: Evolución del costo para input-150 (semilla 337).

7.3. Trayectorias en el mapa

Las figuras 6 y 5 muestran las mejores trayectorias trazadas sobre el mapa, ubicando cada ciudad por sus coordenadas.

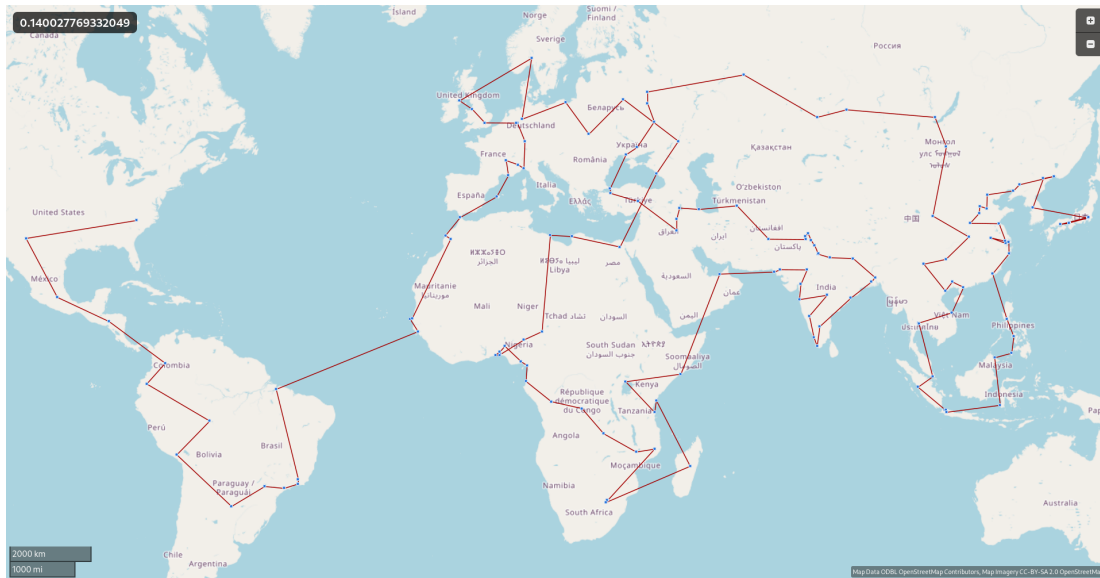


Figura 5: Mejor trayectoria para input-150.



Figura 6: Mejor trayectoria para input-40.



8. Conclusiones

La heurística de Aceptación por Umbrales resultó eficaz para encontrar soluciones factibles a las dos instancias del TSP, alcanzando costos normalizados de 0.2659 y 0.1400. El diseño de la función de costo, que penaliza las aristas inexistentes en lugar de prohibirlas, permitió que la búsqueda convergiera de forma natural hacia soluciones factibles sin necesidad de restricciones explícitas.

La experimentación considero fue lo más importante para poder obtener valores variados y en algún punto llegar a uno lo suficientemente bueno, observamos que la calidad de las soluciones depende fuertemente de los parámetros, en particular del factor de enfriamiento: valores cercanos a 1 producen mejores soluciones a costa de un mayor tiempo de cómputo.



Referencias

- [1] G. B. Dantzig, D. R. Fulkerson, and S. M. Johnson (1954). Solution of a Large-Scale Traveling-Salesman Problem. *Journal of the Operations Research Society of America*, Vol. 2, No. 4, pp. 393–410.
- [2] G. Dueck and T. Scheuer (1990). Threshold Accepting: A General Purpose Optimization Algorithm Appearing Superior to Simulated Annealing. *Journal of Computational Physics*, Vol. 90, No. 1, pp. 161–175.
- [Ianluck] Ianluck. Imágenes de mapas.
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi (1983). Optimization by Simulated Annealing. *Science*, Vol. 220, No. 4598, pp. 671–680.
- [5] S. Lin and B. W. Kernighan (1973). An Effective Heuristic Algorithm for the Traveling-Salesman Problem. *Operations Research*, Vol. 21, No. 2, pp. 498–516.